

Centro Universitário Filadélfia
Polo Salinópolis
Ciência de Dados e Inteligência Artificial

Aula 10

Implantação & Docker

Fundamentos de Propriedade Intelectual Para a Transferência de
Tecnologia de Software

Thiago Rafael da Silva Moura
trsmoura@yahoo.com.br

28 de novembro de 2025



CAPÍTULO 10

IMPLANTAÇÃO & DOCKER

Do "Funciona na minha máquina" para Produção

Manifesto de Carga



1. CONTAINERIZAÇÃO

O fim dos problemas de dependência com Docker.



2. DOCKERFILE

A receita para criar ambientes reprodutíveis.



3. CLOUD DEPLOY

Levando seus containers para a nuvem (PaaS, Serverless).

| O Problema Clássico

"MAS FUNCIONA NA MINHA MÁQUINA!"

Você envia o código para o colega ou para o servidor.

Erro: Biblioteca faltando. Versão do Python diferente. Sistema operacional incompatível (Windows vs Linux).

Ambientes virtuais (`venv`) ajudam, mas não resolvem tudo (ex: drivers de sistema, bancos de dados).



A Solução: Containers

Containers empacotam **tudo** que seu software precisa para rodar:

- » O Código.
- » As Bibliotecas (Pandas, Scikit-Learn).
- » As Ferramentas de Sistema.
- » O Sistema Operacional (uma versão leve).

Garantia: Se roda no container, roda em qualquer lugar.

VM vs Container

MÁQUINA VIRTUAL (VM)

Pesada. Virtualiza o Hardware. Cada VM tem um Sistema Operacional completo (GBs de tamanho).

Lento para iniciar.

CONTAINER

Leve. Virtualiza o Sistema Operacional. Compartilha o kernel do host. MBs de tamanho.

Inicia em milissegundos.

A Metáfora do Container

Antes dos containers de carga, navios carregavam barris, sacos e caixas de tamanhos variados. O carregamento era lento e ineficiente.

O **Container Padrão** revolucionou o comércio global. Não importa o que tem dentro (carros, bananas, eletrônicos), o guindaste sabe como pegar e o navio sabe como encaixar.

O que é Docker?

Docker é a plataforma mais popular para criar e gerenciar containers.

Tornou-se o padrão da indústria para desenvolvimento e deploy de software.

Em Data Science, garante a **reprodutibilidade** total dos experimentos.

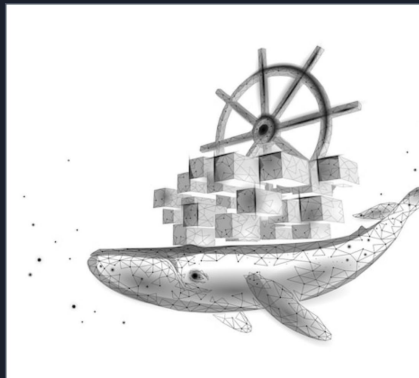


Imagem vs. Container



IMAGEM (IMAGE)

É a **receita** (ou o molde). É estática, somente leitura.

Ex: ``python:3.9-slim``



CONTAINER

É o **bolo** (ou a instância em execução). É criado a partir da imagem.

Você pode ter 10 containers rodando a mesma imagem.

Instalando Docker

Baixe o **Docker Desktop** (Windows/Mac) ou **Docker Engine** (Linux).

Verifique a instalação no terminal:

```
$ docker --version Docker version 20.10.17, build 100c701 $ docker run hello-world Hello from Docker!
```

0 Dockerfile

Um arquivo de texto simples (sem extensão) chamado Dockerfile.

Contém a lista de instruções para o Docker construir sua Imagem.

Vamos construir um Dockerfile passo a passo para uma API de Machine Learning.

1. FROM (A Base)

Todo Dockerfile começa definindo uma imagem base. Não comece do zero!

```
# Use uma imagem oficial do Python FROM python:3.9-slim
```

slim indica uma versão leve (menor tamanho), ideal para produção.

2. WORKDIR & COPY

Definimos onde vamos trabalhar dentro do container e copiamos nossos arquivos.

```
# Define a pasta de trabalho dentro do container WORKDIR /app # Copia o arquivo de requerimentos local para o container
```

3. RUN (Instalação)

Executa comandos dentro do container durante a construção (build).

```
# Instala as dependências Python RUN pip install --no-cache-dir -r requirements.txt
```

Isso instala Pandas, Scikit-Learn, FastAPI, etc., dentro da imagem.

4. CMD (Ação Final)

Define o comando que será executado quando o container **iniciar**.

```
# Comando para iniciar a API CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "80"]
```

Diferença: RUN acontece no Build. CMD acontece no Run.

Construindo a Imagem

Com o Dockerfile pronto, vamos "assá-lo" em uma imagem.

```
# -t dá um nome (tag) para a imagem # . indica que o Dockerfile está na pasta atual $ docker build -t minha-api-m
```

O Docker baixará o Python, copiará os arquivos e instalará as libs. Isso pode levar alguns minutos na primeira vez.

Rodando o Container

Agora podemos criar uma instância (container) da nossa imagem.

```
$ docker run minha-api-ml
```

Mas espere! A API está rodando dentro do container, isolada do mundo. Precisamos abrir uma porta.

Mapeamento de Portas (-p)

Conecta uma porta da sua máquina (Host) a uma porta do Container.

```
# -p HOST:CONTAINER $ docker run -p 8000:80 minha-api-m1
```

Agora, ao acessar localhost:8000 no seu navegador, você acessa a porta 80 dentro do container.

Modo Interativo (-it)

Útil para debugar. Permite entrar dentro do container e rodar comandos.

```
$ docker run -it python:3.9-slim bash root@a1b2c3d4e5f6:/# python Python 3.9.16 (main, Dec 7 2022, 01:00:00)
```

Variáveis de Ambiente (-e)

Nunca coloque senhas no Dockerfile! Injete-as na hora de rodar.

```
$ docker run -e DB_PASSWORD=segredo123 minha-api-m1
```

No Python, você acessa via `os.getenv("DB_PASSWORD")`.

.dockerignore

Funciona igual ao .gitignore.

Impede que arquivos desnecessários sejam copiados para dentro da imagem, mantendo-a pequena e segura.

```
# .dockerignore __pycache__ *.csv .git .env
```

Otimizando Imagens

USE SLIM/ALPINE

Imagens base menores (Slim é mais compatível com Data Science que Alpine).

CAMADAS (LAYERS)

Instale requerimentos *antes* de copiar o código. O Docker faz cache da instalação se o `requirements.txt` não mudar.

MULTI-STAGE

Técnica avançada para descartar ferramentas de build na imagem final.

Docker Compose

E se seu app precisar de um Banco de Dados? Você teria que rodar dois containers e conectá-los manualmente.

O **Docker Compose** permite definir múltiplos containers em um único arquivo YAML.

Comando mágico: `docker-compose up`.

Exemplo docker-compose.yml

```
version: '3' services: api: build: . ports: - "8000:80" redis: image: "redis:alpine"
```

Isso sobe sua API e um banco Redis, conectados automaticamente na mesma rede.

| Orquestração (Kubernetes)

Docker Compose é ótimo para uma máquina. Mas e se você tiver 1000 containers distribuídos em 50 servidores?

Você precisa de um Orquestrador.

Kubernetes (K8s) é o padrão para gerenciar clusters massivos de containers (Google, AWS, Azure).

Estratégias de Cloud

IAAS (INFRASTRUCTURE)

Você aluga a máquina (EC2).
Instala Docker, cuida da segurança.
Controle total, trabalho total.

PAAS (PLATFORM)

Você entrega o Dockerfile/Código. A nuvem roda e escala. (Heroku, Render, App Runner).

SERVERLESS

Você entrega a função. Roda sob demanda. Custo zero se ocioso. (Lambda).

CI/CD: 0 Pipeline Final

Não faça deploy manual do seu laptop!

1. **Push** para GitHub.
2. **GitHub Actions** roda testes.
3. Se passar, constrói imagem Docker.
4. Empurra imagem para registro.
5. Nuvem atualiza o serviço.



| Model Registry

Evite "assar" o modelo (arquivo `.pkl`) dentro da imagem Docker se ele mudar frequentemente.

Boa prática: O container baixa o modelo de um Registro de Modelos (MLflow, S3) ao iniciar.

Isso permite atualizar o modelo sem precisar fazer rebuild do container de software.

Monitoramento em Produção

O software não quebra, mas o modelo "apodrece".

Drift de Dados: A distribuição dos dados de hoje é diferente do treino?

Drift de Conceito: A relação entre X e Y mudou?

Ferramentas: Evidently AI, Arize, Grafana.

Deploy Canário

Não troque o modelo antigo pelo novo de uma vez (Big Bang).

Use **Canary Deployment**:

Envie 10% do tráfego para o Modelo Novo. Se não houver erros e as métricas forem boas, aumente gradualmente para 100%.

Resumo da Jornada

- » ✓ Saímos dos scripts soltos (Notebooks).
- » ✓ Escrevemos código limpo (POO/Funcional).
- » ✓ Garantimos qualidade (Testes, Typing).
- » ✓ Empacotamos em APIs (FastAPI).
- » ✓ Entregamos em Containers (Docker).

VOCÊ AGORA É UM ENGENHEIRO DE ML!

Recursos para Estudo

- » **Livro:** Software Engineering for Data Scientists (Catherine Nelson).
- » **Docker:** documentação oficial (docs.docker.com).
- » **MLOps:** curso "Made With ML".
- » **FastAPI:** fastapi.tiangolo.com.



EOF (END OF FILE)

Obrigado pela atenção!

Dúvidas?

FIM